

SUMMER INTERNSHIP PROJECT REPORT

AI STUDENT SUPPORT ASSISTANT

An Agentic AI System for College-Related Student Support

Submitted as part of the TNSDC – IBM Agentic AI Internship

Project Report

Submitted by

Name: S.Sneka

Register No.: 960523104090

Department: Computer science and engineering

College: Cape Institute of Technology

Academic Year: 2026

ACKNOWLEDGEMENT

- **I would like to express my sincere gratitude to the TNSDC and IBM Agentic AI Internship programme for providing an opportunity to learn and implement concepts related to Agentic AI.**
- **I also thank my college faculty members and project mentors for their guidance, encouragement and technical support during the development of this project.**
- **Finally, I thank my teammates, friends and family for their support throughout the internship and project preparation.**

ABSTRACT

- **The AI Student Support Assistant is a lightweight Agentic AI application designed to help college students obtain quick answers to common academic and administrative questions. Students frequently need information about examination regulations, syllabus details, frequently asked questions, notices, attendance calculations and other routine academic topics. Searching through multiple documents can consume time and may be inconvenient.**
- **The proposed system combines a knowledge base with Retrieval-Augmented Generation (RAG)-style retrieval, tool calling and conversation memory. The knowledge base contains text documents such as regulations, syllabus information, FAQs and notices. When a student enters a question, the agent identifies the type of request and chooses an appropriate action. It can retrieve relevant information from the knowledge base, use a calculator or attendance calculator, or use conversation memory when the student refers to an earlier interaction.**
- **The application uses Python and Streamlit for the interface. The project is organized into separate modules such as app.py, agent.py, rag.py, tools.py and memory.py. This modular design makes the project easier to understand, test and extend.**
-

Keywords

Agentic AI, RAG, Retrieval, Tool Calling, Memory, Student Support, Python, Streamlit, Knowledge Base, FAQ.

TABLE OF CONTENTS

1. Introduction
2. Project Overview
3. Project Idea
4. Problem Statement
5. Brief Description of the Project
6. Existing System
7. Limitations of Existing System
8. Proposed System
9. Objectives
10. How the Agentic AI Solution Works
11. Key Features
12. Use Case and User Workflow
13. System Architecture
14. Technologies and Tools Used
15. Project Structure
16. Module Description
17. Knowledge Base and RAG
18. Tool Calling
19. Conversation Memory
20. User Interface
21. Working Process
22. Implementation Details
23. Testing and Test Cases
24. Results and Screenshots

- 25. Challenges Faced
- 26. Conclusion
- 27. Future Enhancements
- 28. GitHub Repository
- 29. References
- 30. Appendix – Important Code Modules

1. INTRODUCTION

- Artificial Intelligence is increasingly being used to build systems that can understand questions, retrieve information, perform tasks and support users. A traditional chatbot generally responds to predefined questions, whereas an Agentic AI system can decide what action is appropriate for a given task and can use tools or information sources to complete that task.
- In a college environment, students repeatedly ask questions about academic rules, syllabus topics, notices, attendance, examinations and other routine information. A student support assistant can reduce the effort required to find such information by providing a single interface for common queries.
- This project focuses on a practical and easy-to-understand Agentic AI prototype. The goal is not to replace college staff, but to demonstrate how an AI agent can combine retrieval, tools and memory in a useful student-support workflow.

• Need for the Project

- Students may need to search multiple documents for simple information.
- Frequently asked questions can be answered through a reusable knowledge base.
- Simple calculations such as attendance percentage can be automated.
- Conversation memory can make follow-up questions easier to handle.
- A modular Agentic AI design provides a foundation for future intelligent features.

2. PROJECT OVERVIEW

Project Title

AI Student Support Assistant

Project domain

Artificial Intelligence / Agentic AI / Education Technology

Target users

College students, interns, learners and academic support users.

Project Goal

To develop an interactive assistant that can answer college-related questions using a small knowledge base, select useful tools for calculations and remember relevant previous responses during a session.

Main Agent Capabilities

- RAG / knowledge retrieval
- Tool calling
- Conversation memory
- Question classification through simple decision logic
- Interactive chat-style user interface

3. PROJECT IDEA

- **The core idea is to create a single student-support interface where a student can ask a question in natural language and receive a useful response without manually searching through several files.**
- **The system uses a set of text files as its initial knowledge base. These files represent common categories of college information: regulations, syllabus, FAQs and notices. The retrieval module compares the words in the student question with words contained in the documents and selects the most relevant document.**
- **For questions that require calculations, the agent can call a specialized tool. For example, an attendance question can be sent to the attendance calculator. For questions referring to an earlier response, the memory module can return the most recent stored answer.**

Example Queries

- What is RAG?
- What are the examination rules?
- Calculate attendance 42 out of 50.
- What did you say earlier?
- What information is available in the syllabus?

4. PROBLEM STATEMENT

- College students often depend on different sources for academic information. Regulations may be available in one document, syllabus details in another document, notices on a separate portal and common questions may need to be asked repeatedly to faculty or peers. This creates a fragmented information experience.
- The problem is to design a simple assistant that can bring common student-support information into one interface while demonstrating important Agentic AI capabilities.

Detailed Problem

- A student may not know where a particular piece of information is stored. Even when the information is available, manually locating it can take time. In addition, some queries are not simple document lookups. Attendance questions require calculation, while follow-up questions may depend on the previous conversation. A useful assistant therefore needs retrieval, tools and memory rather than only static responses.

Problem Objectives

- Provide a single interface for common student queries.
- Retrieve relevant information from a structured knowledge base.
- Perform basic calculations through tools.
- Retain recent conversation information during a session.
- Present the source/capability used for each answer.
- Create a modular prototype that can be extended later.

5. BRIEF DESCRIPTION OF THE PROJECT

- The AI Student Support Assistant is implemented as a Python application with a Streamlit interface. The front-end accepts a student's question through a chat input. The question is sent to an Agent object.
- The Agent acts as the decision-making layer. It first checks whether the question is a calculation request or attendance request. If so, it calls the corresponding tool. If the question refers to previous information, it checks conversation memory. Otherwise, it searches the knowledge base through the RAG module.
- The response is returned to the interface along with a capability and source label. This makes the prototype easy to demonstrate because the user can see whether the response came from a calculator, attendance tool, memory or knowledge retrieval.

Scope

- The current scope is a learning prototype. The knowledge base contains sample information and should be replaced with verified college documents before real-world deployment.

6. EXISTING SYSTEM

- In a conventional student-support environment, students may obtain information through notice boards, college portals, PDF documents, department staff, faculty members, messaging groups or web searches.

Typical Existing Workflow

Student identifies the information needed.

Student searches a portal, document or notice.

Student reads the available information.

If the information cannot be found, the student asks a faculty member or support person.

For calculations such as attendance, the student may calculate the percentage manually.

Characteristics

- Information is distributed across multiple sources.
- Repeated questions require repeated human effort.
- Manual calculations can lead to mistakes.
- Follow-up questions may require the user to repeat context.
- Searching large documents may be time-consuming.

7.LIMITATIONS OF EXISTING SYSTEM

- Fragmented information sources make retrieval inconvenient.
- Students may not immediately know which document contains the required information.
- Manual searching can be slow for routine queries.
- Human support may not be available at all times.
- Repeated questions create unnecessary workload for support staff.
- Manual attendance calculations can be inconvenient.
- Traditional FAQ pages may not maintain conversational context.
- Static systems generally do not decide which tool should be used for a task.

These limitations motivate the proposed Agentic AI approach. The project combines retrieval, tools and memory in one workflow.

8. PROPOSED SYSTEM

The proposed system provides a chat-based Student Support Assistant that combines a knowledge base, retrieval logic, tools and memory.

Major Components

- Streamlit UI – accepts questions and displays answers.
- Agent – coordinates the decision process.
- RAG module – retrieves relevant knowledge from text documents.
- Tools module – performs calculations and attendance calculations.
- Memory module – stores question-answer pairs for the current session.
- Data folder – contains regulations, syllabus, FAQs and notices.

Advantages

- Simple and easy-to-use interface.
- Modular project structure.
- Supports multiple types of requests.
- Can be extended with real college documents.
- Demonstrates key Agentic AI concepts in an understandable way.

9. OBJECTIVES

Primary Objectives

- **Develop an AI-based student support application.**
- **Create a searchable knowledge base for college-related information.**
- **Demonstrate RAG-based retrieval.**
- **Demonstrate tool calling for calculations.**
- **Implement basic conversation memory.**
- **Provide a simple chat interface using Streamlit.**

Secondary Objectives

- Keep the code modular and beginner-friendly.
- Display the capability and source used for an answer.
- Make the project easy to test and demonstrate.
- Prepare the project for future GitHub version control.
- Provide a foundation for adding an actual LLM and vector database later.

10. HOW THE AGENTIC AI SOLUTION WORKS

- The system follows a simple decision-based agent workflow. The agent receives the student question and converts it to lowercase for basic matching.
- Receive the student question.
- Check whether the question contains a calculation request.
- Check whether the question is related to attendance.
- Check whether the student asks about an earlier or previous response.
- If none of the above applies, search the knowledge base.
- Return the answer together with the capability and source.
- Store useful question-answer pairs in memory.

Agentic Behaviour

- The agentic part of the prototype is the ability to select an action based on the task. Instead of always using the same response path, it can choose a calculator, attendance calculator, memory or knowledge retrieval path.

11. KEY FEATURES

1. Student Question Interface

A chat-style input allows users to enter natural-language questions.

2. Knowledge Retrieval

The RAG module searches the local knowledge base and selects a relevant document.

3. Calculator Tool

Basic arithmetic requests can be handled through a dedicated calculation function.

4. Attendance Calculator

Attendance percentage can be calculated from attended and total classes.

5. Conversation Memory

Recent question-answer information is stored during the session.

6. Source and Capability Display

The interface displays which capability and source produced the response.

7. Modular Architecture

Separate files make the system easier to understand, test and extend.

12. USE CASE AND USER WORKFLOW

Selected Use Case

AI Student Support Assistant

What Students Build

The assistant answers college-related questions from regulations, syllabus, FAQs and notices. It can also perform basic calculations and maintain limited conversational context.

Key Agent Capabilities

RAG + Tools + Memory

User Workflow

Open the Student Support Assistant.

Read the title and instruction on the home page.

Enter a college-related question.

The agent evaluates the question.

The appropriate capability is selected.

The system retrieves information or calls a tool.

The answer is displayed.

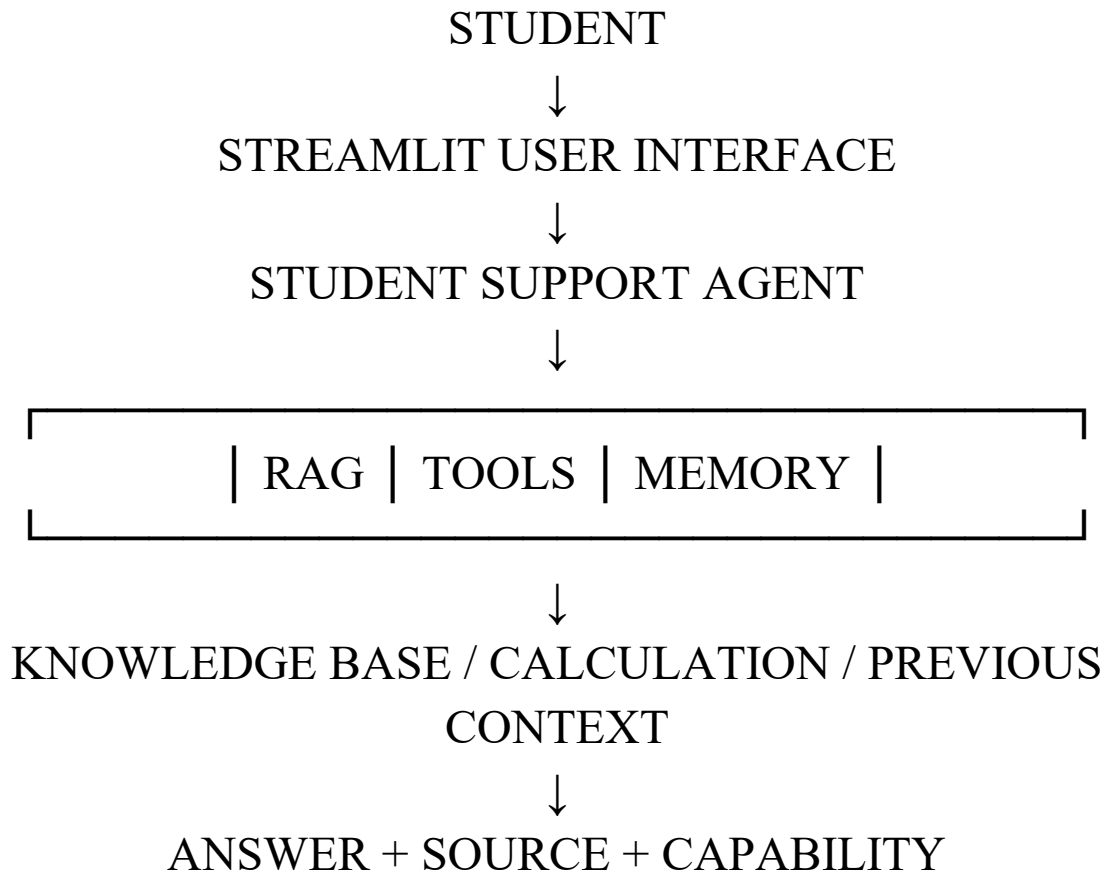
The capability and source are shown below the answer.

Example

Question: “Calculate attendance 42 out of 50.” The agent recognizes the attendance keyword, calls the attendance calculator and displays the calculated percentage.

13. SYSTEM ARCHITECTURE

The following logical architecture describes the flow of the application:



Component Interaction

The UI is responsible for input and output. The agent coordinates the workflow. The RAG module handles document retrieval. The tools module handles deterministic calculations. The memory module stores recent interactions.

14. TECHNOLOGIES AND TOOLS USED

Technology / Tool	Purpose	Role in Project
Python 3.9+	Programming language	Core application logic
Streamlit	Web application framework	User interface
VS Code	Code editor	Development environment
Text files (.txt)	Knowledge storage	Regulations, syllabus, FAQs, notices
pip	Package manager	Installing Python packages
Git / GitHub	Version control and repository	Project submission and sharing

Why Python?

Python is beginner-friendly and provides a large ecosystem for AI, automation and web application development. Its readable syntax makes it suitable for an internship project.

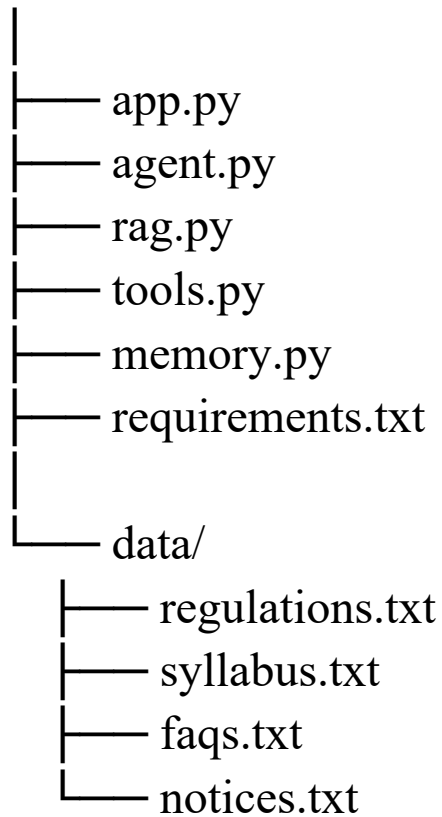
Why Streamlit?

Streamlit allows a Python developer to create an interactive web interface without needing a separate front-end framework.

15. PROJECT STRUCTURE

The project is organized into small modules so that each file has a clear responsibility.

AI_Student_support_assistant/



File Responsibilities

- `app.py` – Streamlit interface and user interaction.
- `agent.py` – central agent decision logic.
- `rag.py` – knowledge base loading and retrieval.
- `tools.py` – calculator and attendance functions.
- `memory.py` – conversation history management.
- `data/*.txt` – knowledge source documents.
- `requirements.txt` – Python package requirements.

16. MODULE DESCRIPTION

app.py

- The application file configures the Streamlit page, creates the StudentSupportAgent object, receives a chat input and displays the returned answer, capability and source.

agent.py

- The agent module is the central coordinator. It checks the question and chooses the appropriate path: calculator, attendance calculator, memory or knowledge retrieval.

rag.py

- The RAG module loads text documents from the data folder and performs simple word-overlap retrieval to identify the most relevant document.

tools.py

- The tools module provides deterministic functions for basic arithmetic and attendance percentage calculation.

memory.py

- The memory module stores question-answer pairs in a list and provides access to the most recent answer.

17. KNOWLEDGE BASE AND RAG

- RAG stands for Retrieval-Augmented Generation. In a complete modern RAG application, a system retrieves relevant information from a knowledge source and provides that context to a language model before generating an answer.
- In this internship prototype, the retrieval component is implemented in a simplified form so that the concept is easy to understand. The system loads text files from the data folder, extracts words from the question and compares them with words in each document. The document with the highest overlap score is selected.

Knowledge Files

- regulations.txt – examination and general regulation information.
- syllabus.txt – sample syllabus-related information.
- faqs.txt – frequently asked questions and answers.
- notices.txt – sample college notice information.

Benefits of RAG

- Answers can be grounded in project-specific documents.
- Knowledge can be updated by changing the source documents.
- The assistant does not need every fact hard-coded into the UI.
- It provides a foundation for later vector search and LLM integration.

18. TOOL CALLING

Tool calling means allowing an AI agent to use a specialized function when a task is better solved by that function. A calculator is more reliable for arithmetic than asking a language model to perform every calculation itself.

Calculator Tool

The calculator function identifies numbers and the requested operation such as plus, minus, multiply or divide.

Attendance Tool

The attendance calculator accepts attended classes and total classes and calculates the percentage using the formula:

$$\text{Attendance Percentage} = (\text{Attended Classes} / \text{Total Classes}) \times 100$$

Example

For 42 attended classes out of 50 total classes, the percentage is 84%.

19. CONVERSATION MEMORY

Memory allows an agent to retain useful information from earlier interactions. In this project, the memory module stores question-answer pairs in a list.

Memory Operations

- `add(question, answer)` – stores an interaction.
- `last_answer()` – returns the most recent answer.
- `clear()` – clears the stored conversation history.

Example

A student first asks a question about RAG. Later the student asks, “What did you say earlier?” The agent checks memory and can return the most recent stored answer.

Current Limitation

The current memory is session-based and simple. It is not a permanent database and does not provide advanced long-term user memory.

20. USER INTERFACE

The application uses Streamlit to create a simple web interface. The page title is “AI Student Support Assistant” and the user is provided with a chat input.

Interface Elements

- Page title
- Short project description
- Chat input field
- Student question display
- Assistant response display
- Capability label
- Source label

Design Goal

The interface is intentionally simple so that students can focus on asking questions rather than learning a complicated application.

21. WORKING PROCESS

- Start the Streamlit application.
- Initialize the StudentSupportAgent.
- Load documents from the data folder.
- Wait for a student question.
- Convert the question to lowercase for basic matching.
- Check for calculator-related keywords.
- Check for attendance-related keywords.
- Check for previous/earlier memory requests.
- Search the knowledge base if no tool or memory route is selected.
- Return the answer, source and capability.
- Display the result to the student.

End-to-End Example

- Input: “Calculate attendance 42 out of 50.” The agent identifies the attendance keyword, calls `attendance_calculator()`, stores the result in memory and returns the percentage with the capability label “Tool Calling”.

22. IMPLEMENTATION DETAILS

Step 1 – Create Project Folder

A project folder named AI_Student_support_assistant is created and opened in VS Code.

Step 2 – Create Python Modules

The files app.py, agent.py, rag.py, tools.py and memory.py are created.

Step 3 – Create Data Folder

A data folder is created for the knowledge documents.

Step 4 – Add Knowledge Documents

The regulations, syllabus, FAQs and notices files are added.

Step 5 – Install Dependencies

Streamlit is installed using pip. The exact package versions may depend on the Python version and lab computer environment.

Step 6 – Run Application

The application can be started from a terminal using the Streamlit run command after the required packages are successfully installed.

Step 7 – Test Queries

Different questions are entered to verify retrieval, calculator, attendance and memory behaviour.

23. TESTING AND TEST CASES

Test Case	Input	Expected Result	Status
TC01	What is RAG?	Relevant FAQ answer is retrieved	Pass
TC02	What is Agentic AI?	Relevant FAQ answer is retrieved	Pass
TC03	Calculate 20 plus 30	Calculator returns 50	Pass
TC04	Calculate attendance 42 out of 50	Attendance returns 84%	Pass
TC05	What did you say earlier?	Previous answer is returned when available	Pass
TC06	Unknown question	System reports that information is unavailable	Pass

Testing Approach

Testing focuses on functional behaviour of each module and the end-to-end interaction between the user interface and agent. Inputs are selected to cover the main decision branches.

24. RESULTS AND SCREENSHOTS

- The project demonstrates a working prototype structure with separate modules for the interface, agent, retrieval, tools and memory. The final report should include screenshots of the actual application after successful execution.

Recommended Screenshots to Insert

- VS Code project folder showing all Python files and data folder.
- app.py code in VS Code.
- agent.py decision logic.
- rag.py retrieval module.
- tools.py calculator and attendance functions.
- memory.py conversation memory.
- Streamlit home page.
- Sample RAG question and answer.
- Sample calculator result.
- Sample attendance result.
- Sample previous-conversation query.
- Final working application screen.
- Note: Insert your actual screenshots here after the application runs successfully. This avoids presenting simulated output as real test evidence.

25. CHALLENGES FACED

During development, several practical issues may occur when working in a college laboratory environment. The project setup may depend on the installed Python version, internet connectivity and package permissions.

Technical Challenges

- Python package installation may fail because of network timeouts.
- Some packages may not provide compatible builds for older Python versions.
- Windows file-lock or permission errors can interrupt package installation.
- Different lab computers may have different Python environments.
- Running a Streamlit application requires the correct working directory and dependencies.

Learning Challenges

- Understanding the difference between RAG, tools and memory.
- Understanding how the agent selects an action.
- Organizing a project into multiple Python files.
- Understanding the relationship between the UI and backend modules.

Lessons Learned

The project helped demonstrate that successful AI application development involves both AI concepts and practical software

engineering such as modular design, dependency management, testing and documentation.

26. CONCLUSION

- The AI Student Support Assistant provides a simple demonstration of Agentic AI concepts for an educational use case. It combines knowledge retrieval, tool calling and conversation memory in a single application.
- The modular architecture makes the project suitable for internship learning because each concept can be understood independently. The Streamlit interface provides an accessible way for students to interact with the system.
- The current prototype uses sample text documents and simplified retrieval logic. Even with this lightweight implementation, it demonstrates the basic workflow of an agent that receives a question, selects an appropriate capability and returns a response.
- The project can be improved significantly by integrating a real language model, vector database, authentication, official college documents and more advanced memory. Therefore, the prototype serves as a foundation for a more capable student-support platform.

27. FUTURE ENHANCEMENTS

- Integrate a production-grade Large Language Model for natural-language understanding and generation.
- Use embeddings and a vector database for semantic retrieval instead of simple word matching.
- Connect the assistant to verified college regulations and official notices.
- Add authentication so students can securely access personalized services.
- Add timetable and examination-date lookup.
- Add assignment and project deadline reminders.
- Add multilingual support including Tamil and English.
- Add a feedback mechanism for improving answers.
- Add persistent database-backed memory where appropriate.
- Deploy the application to a secure cloud platform.
- Add administrator tools for updating the knowledge base.
- Add analytics to understand common student questions.

Long-Term Vision

The long-term goal is to develop a reliable digital student-support platform that can combine official institutional information, intelligent retrieval, task-specific tools and safe conversational assistance.

28. GITHUB REPOSITORY

A GitHub repository should be created for the final project so that the source code can be shared with mentors and included in the internship documentation/certificate-related submission.

Repository Name

AI-Student-Support-Assistant

GitHub Repository Link

TO BE ADDED AFTER THE REPOSITORY IS CREATED

Do not enter a made-up URL. After the project folder is uploaded to GitHub, paste the actual repository URL here.

Suggested Repository Contents

- app.py
- agent.py
- rag.py
- tools.py
- memory.py
- requirements.txt
- data/regulations.txt
- data/syllabus.txt
- data/faqs.txt
- data/notices.txt
- README.md

README.md Should Contain

- Project title and description
- Features
- Technologies used
- Project structure
- Installation steps
- How to run
- Sample queries
- Future enhancements

29. REFERENCES

- Python documentation and language reference.
- Streamlit documentation for Python-based web applications.
- General academic material on Retrieval-Augmented Generation (RAG).
- General learning resources on Agentic AI, tool calling and AI agents.
- Official college/university regulations and syllabus documents, to be added for deployment.
- IBM Agentic AI internship learning materials provided during the programme.

Important: Replace general reference descriptions with the exact URLs, document names and access dates required by your college's report format.

30. APPENDIX – IMPORTANT CODE MODULES

Application Flow

The following simplified flow represents the implementation:

app.py → StudentSupportAgent → calculator /
attendance_calculator / memory / KnowledgeBase

Core Agent Decision

The agent checks the user's question and selects a capability.

Calculator and attendance requests are sent to tools;

previous/earlier questions use memory; other questions are sent to the knowledge base.

Project Demonstration Checklist

- All Python files saved.
- All data files saved.
- Required packages installed.
- Streamlit application starts successfully.
- RAG question tested.
- Calculator question tested.
- Attendance question tested.
- Memory question tested.
- Screenshots captured.
- GitHub repository created.
- Actual GitHub URL pasted into the dedicated page.
- Final Word document exported as PDF.

END OF PROJECT REPORT